第四章 数据处理

2019-4-28

计算物理基础

本章内容

§4.1 插值

§4.2 曲线拟合

§4.3 曲线拟合工具箱(阅读)

§4.4 求方程的零点

§4.5 快速傅里叶变换



学习指导

1. 掌握用插值法和曲线拟合法处理数据。
2. 掌握求函数的零点与极大极小值。
3. 学习快速傅里叶变换。
4. 了解曲线拟合工具箱的用法与功能。

第四章指令汇总

插值（interp1, interp2, interp3, …）

拟合（polyfit, polyval, cftool）

找零点（fzero, fsolve, solve, roots）

找极小值（fminbnd, fminsearch）

快速傅里叶变换FFT

(fft, ifft, fft2, ifft2, fftn, ifftn)

§4.1.1 插值分类

按插值范围	按插值方法
• 内插 • 外插（外推）	• 多项式插值 • 样条插值 • 曲线拟合 • 分段插值

§4.1.2 拉格朗日插值法

过两点$(x_0,y_0),(x_1,y_1)$直线 $$y=\frac{x-x_1}{x_0-x_1}y_0+\frac{x-x_0}{x_1-x_0}y_1$$

令 $$L_1(x)=y, l_0=\frac{x-x_1}{x_0-x_1}, l_1=\frac{x-x_0}{x_1-x_0}$$

$$L_1(x)=l_0(x)y_0+l_1(x)y_1$$

显然

$$l_0(x_0)=1,\quad l_0(x_1)=0,\quad l_1(x_0)=0,\quad l_1(x_1)=1$$

$l_0(x), l_1(x)$：对应节点x_0, x_1的线性插值基函数。

插值函数：两个插值基函数的线性组合，组合系数为对应点上的函数值y_0, y_1。

二次插值函数

令

$$L_2(x) = l_0(x)y_0 + l_1(x)y_1 + l_2(x)y_2$$

$l_0(x), l_1(x), l_2(x)$：插值基函数，不超过二次的多项式。

且满足条件

$$l_0(x_0) = 1,\ l_0(x_1) = 0,\ l_0(x_2) = 0$$
$$l_1(x_0) = 0,\ l_1(x_1) = 1,\ l_1(x_2) = 0$$
$$l_2(x_0) = 0,\ l_2(x_1) = 0,\ l_2(x_2) = 1$$

即

$$l_k(x_i) = \delta_{ik}(i, k = 0,1,2)$$

$$\text{其中}\delta_{ik} = \begin{cases} 1, i = k \\ 0, i \neq k \end{cases}$$

北京师范大学

构造基函数

$l_0(x)$ 是二次多项式

含因子$(x-x_1)(x-x_2)$

$$l_0(x) = A(x-x_1)(x-x_2)$$

A由条件 $l_0(x_0)=1$定出

$$A = \frac{1}{(x_0-x_1)(x_0-x_2)}$$

结论

$$l_0(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}$$

北京师范大学

同理

$$l_1(x)=\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}$$

$$l_2(x)=\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

得到

$$L_2(x)=\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0+\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1+\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2$$

$L_2(x)$:

- 是一个不超过二次的多项式。
- 满足插值条件$L_2(x_0)=y_0,\ L_2(x_1)=y_1,\ L_2(x_2)=y_2$。

一般情况

给定函数$y=f(x)$在$n+1$个节点的函数值 → 求出一个不超过n次的插值多项式

$$l_k(x)=\frac{(x-x_0)\cdots(x-x_{k-1})(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_0)\cdots(x_k-x_{k-1})(x_k-x_{k+1})\cdots(x_k-x_n)}$$

$$L_n(x)=\sum_{k=0}^{n} l_k(x)y_k$$

满足条件：

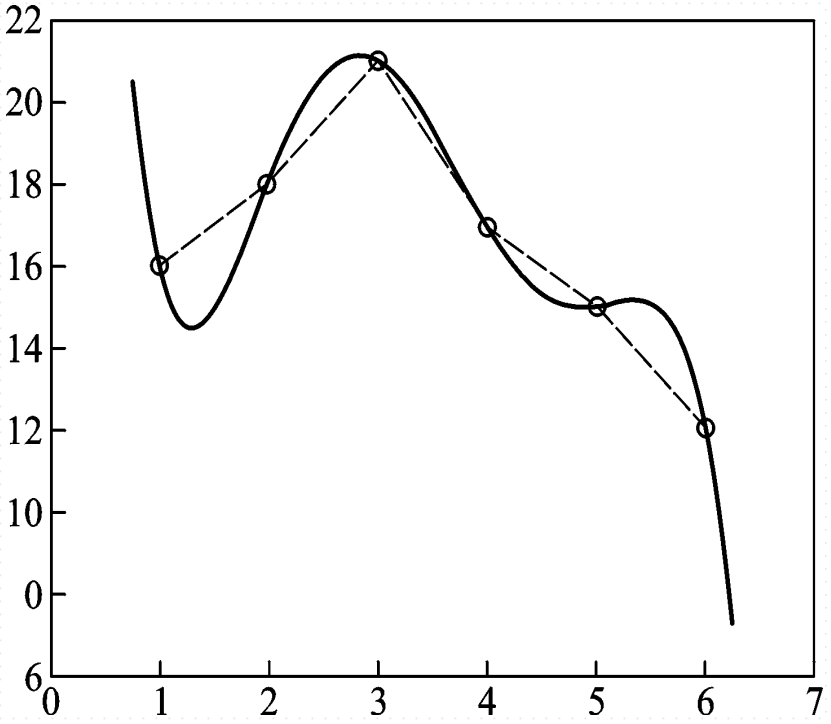
$l_k(x_i)=\delta_{ik}$

编写程序

```
function v = polyinterp(x,y,u)
    n = length(x);
    v = zeros(size(u));
    for k = 1:n
       w = ones(size(u));
       for j = [1:k-1 k+1:n]
       w = (u-x(j))./(x(k)-x(j)).*w;
       end
       v = v + w*y(k);
end
```

应用实例

```
x = 1:6;
y = [16,18,21,17,15,12];
u= 0.75:0.05:6.25;
v=polyinterp(x,y,u)
plot(x,y,':',u,v,'-')
```



- x，y为已知数据
- u是要插入的数据

- 圆圈：原数据点
- 实线：插值曲线
- 虚线：分段线性插值

结果分析

拉格朗日插值法：曲线有无限多阶导数
分段线性插值法：数据点处发生转折

→ 插值方法的两个极端，需要改进

改进思路

- 分段插值：将数据分成小区域插值。
- 埃尔米特插值：使分段曲线光滑连接，即连接点有一阶甚至二阶导数。

- 不同方法对导数的限制条件不同，指令pchip和spline是两种方法。

§4.1.3 分段三次埃尔米特插值

将插值区间分为n个小区间，第k个小区间：$h_k = x_{k+1} - x_k$

插入三次多项式：$P(x) = a_3x^3 + a_2x^2 + a_1x + a_0$

用$y_k, y_{k+1}, d_k, d_{k+1}$表示这个多项式：

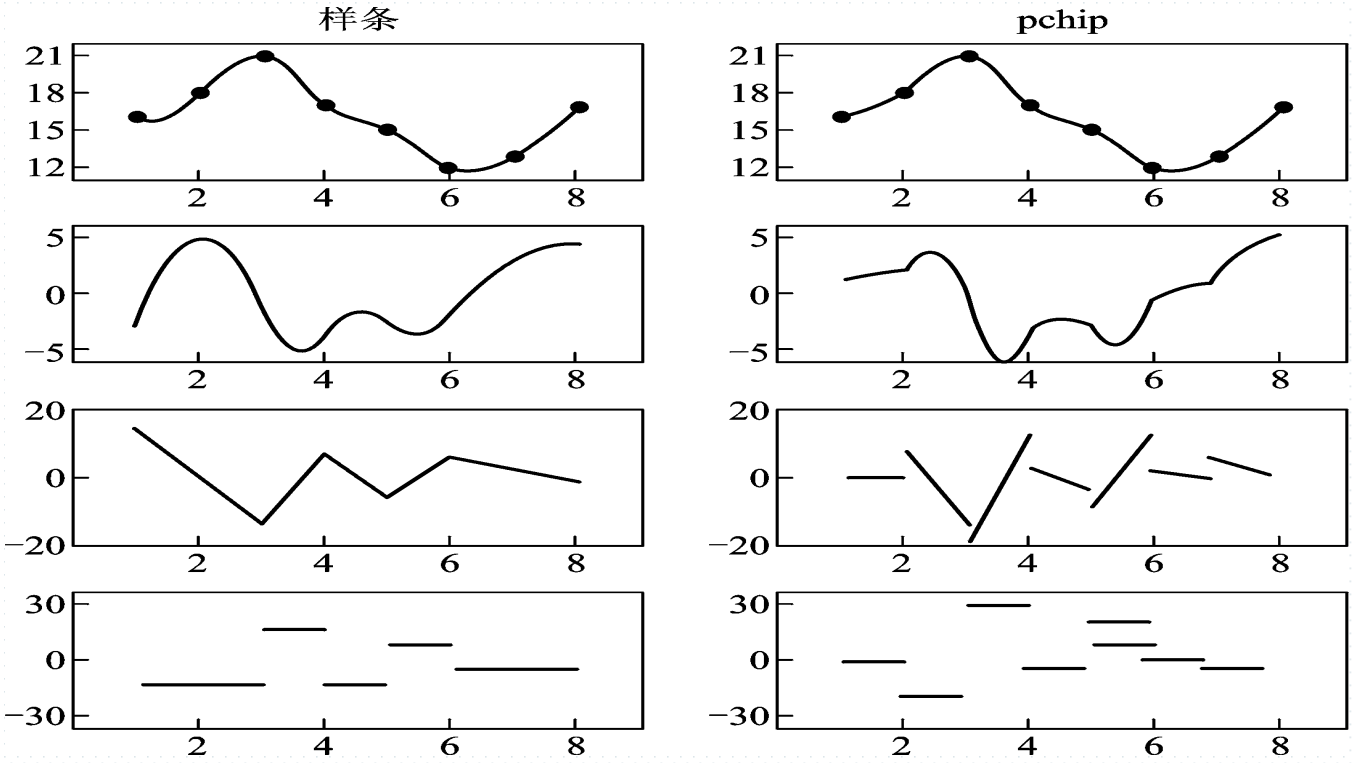
$$P(x) = \frac{3hs^2 - 2s^3}{h^3} y_{k+1} + \frac{h^3 - 3hs^2 + 2s^3}{h^3} y_k + \frac{s^2(s-h)}{h^2} d_{k+1} + \frac{s(s-h)^2}{h^2} d_k$$

- 其中引入了局部变量$s = x - x_k$，令$h = h_k$。当$x = x_k$时有$s = 0$，当$x = x_{k+1}$时满足

$$P(x_k) = y_k, P(x_{k+1}) = y_{k+1}$$
$$P'(x_k) = d_k, P'(x_{k+1}) = d_{k+1}$$

端点导数值选择

- 一阶导数相等：保形分段三次插值（pchip）的做法。（ pchip：分段三次埃尔米特插值多项式）
- 一阶和二阶导数都相等：三次样条插值函数(cubic spline)的做法。



§4.1.4 MATLAB插值指令

Matlab插值指令用法

- 上面介绍的插值方法都可以用指令完成。
- 指令根据输入参数来决定所用插值方法。
- 指令可以进行二维三维以至高维的插值。

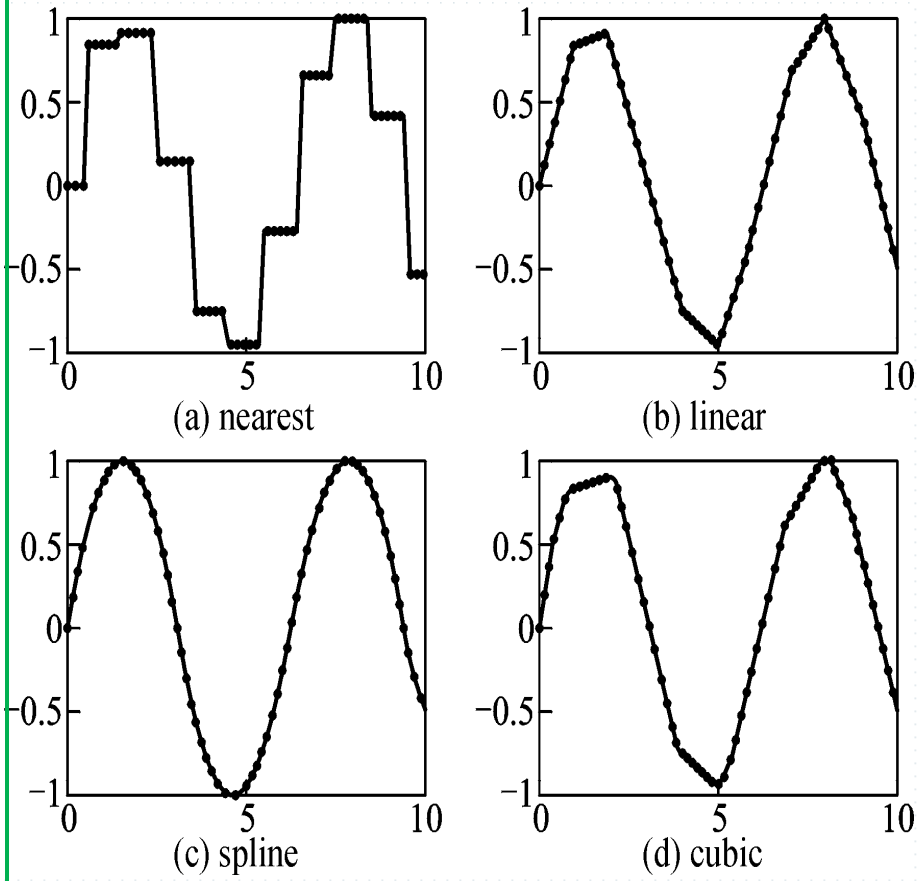
interp1(X,Y,XI,'method')	一维插值
interp1q(X,Y,XI)	快速一维插值
interp2(X,Y,Z,XI,YI)	二维插值
interp3(X,Y,Z,V,XI,YI,ZI)	三维插值指令
interpn(X1,X2,X3,...,V,Y1,Y2,Y3,…)	N维插值指令

参数介绍

- X,Y,Z,V：已知的数据点与函数值
- XI,YI,ZI：插入的数据点
- Method：选用的插值方法
  - nearest　最近邻插值
  - linear　线性插值
  - spline　分段三次样条插值
  - pchip　保形分段三次插值
  - cubic　与pchip相同

例 以正弦函数的10个值作为原始数据，用四种方法算出41个插值点并画曲线。

```
x=0:10; y=sin(x); %原始数据
xi=0:.15:10;      %插值点
yi1=interp1(x,y,xi,'nearest');
yi2=interp1(x,y,xi,'linear');
yi3=interp1(x,y,xi,'spline');
yi4=interp1(x,y,xi,'cubic');
subplot(2,2,1)
plot(xi,yi1,xi,yi1,'r.')
subplot(2,2,2)
plot(xi,yi2,xi,yi2,'r.')
subplot(2,2,3)
plot(xi,yi3,xi,yi3,'r.')
subplot(2,2,4)
plot(xi,yi4,xi,yi4,'r.')
```



§4.2 曲线拟合

§4.2.1 曲线拟合的最小二乘法

曲线拟合：也叫函数逼近，是另一种插值方法。

插值法
- 要求函数通过数据点
- 函数与数据点偏差为零

曲线拟合
- 不要求函数通过数据点
- 可能存在偏差，反映逼近程度

设在曲线拟合中，插值函数值$\varphi(x_i)$与n个原始数据y_1偏差为

$$\delta_i = \varphi(x_i) - y(i = 0,1,2,\ldots n)$$

判断准则

1). 使偏差的绝对值之和为最小：$\sum_{i=0}^{n} |\delta_i|$

2). 偏差的最大绝对值为最小(最佳一致逼近函数)

$$\max_i |\delta_i| = \min$$

3). 使偏差的平方和为最小（最小二乘法）：$\sum_{i=0}^{n} \delta_i^2$

或要求加权的偏差的平方和为最小：$\sum_{i=0}^{n} \lambda_i \delta_i^2$

§4.2.2 多项式拟合

- 设所求插值函数是一个m次多项式$P_m(x),(m<n)$

$$P_m(x)=a_0+a_1x+\mathrm{L}\ +a_mx^m=\sum_{j=0}^{m}a_jx^j$$

- 选取多项式的系数a_j使得在x_i,y_i点有

$$\sum_{i=1}^{n}\delta_i^2=\sum_{i=1}^{n}[y_i-P_m(x_i)]^2==F(a_0,a_1,\mathrm{L}\ ,a_m)$$

- 最小。由多元函数取极值的条件得

$$\frac{\partial F}{\partial a_j}=-2\sum_{i=1}^{n}\left[y_i-\sum_{k=0}^{m}a_kx_i^k\right]x_i^j=0,$$

- 移项得方程组 $\sum_{k=0}^{m} a_k \left(\sum_{i=1}^{n} x_i^{k+j} \right) = \sum_{i=1}^{n} y_i x_i^j$

对于不同i，即

$$\begin{cases} na_0 + a_1 \sum_{i=1}^{n} x_i + a_2 \sum_{i=1}^{n} x_i^2 + \mathrm{L} + a_m \sum_{i=1}^{n} x_i^m = \sum_{i=1}^{n} y_i \\ a_0 \sum_{i=1}^{n} x_i + a_1 \sum_{i=1}^{n} x_i^2 + a_2 \sum_{i=1}^{n} x_i^3 + \mathrm{L} + a_m \sum_{i=1}^{n} x_i^{m+1} = \sum_{i=1}^{n} y_i x_i \\ \cdots\cdots \\ a_0 \sum_{i=1}^{n} x_i^m + a_1 \sum_{i=1}^{n} x_i^{m+1} + a_2 \sum_{i=1}^{n} x_i^{m+2} + \mathrm{L} + a_m \sum_{i=1}^{n} x_i^{2m} = \sum_{i=1}^{n} y_i x_i^m \end{cases}$$

- 当最高幂次为1时就是最小二乘法的拟合直线，正则方程组简化为

$$\begin{cases} na_0 + a_1 \sum_{i=1}^{n} x_i = \sum_{i=1}^{n} y_i \\ a_0 \sum_{i=1}^{n} x_i + a_1 \sum_{i=1}^{n} x_i^2 = \sum_{i=1}^{n} y_i x_i \end{cases}$$

§4.2.3 多项式拟合指令polyfit

MATLAB将多项式拟合开发成专用的指令，在指令的程序中使用了很先进的算法来求解正则方程。

算法

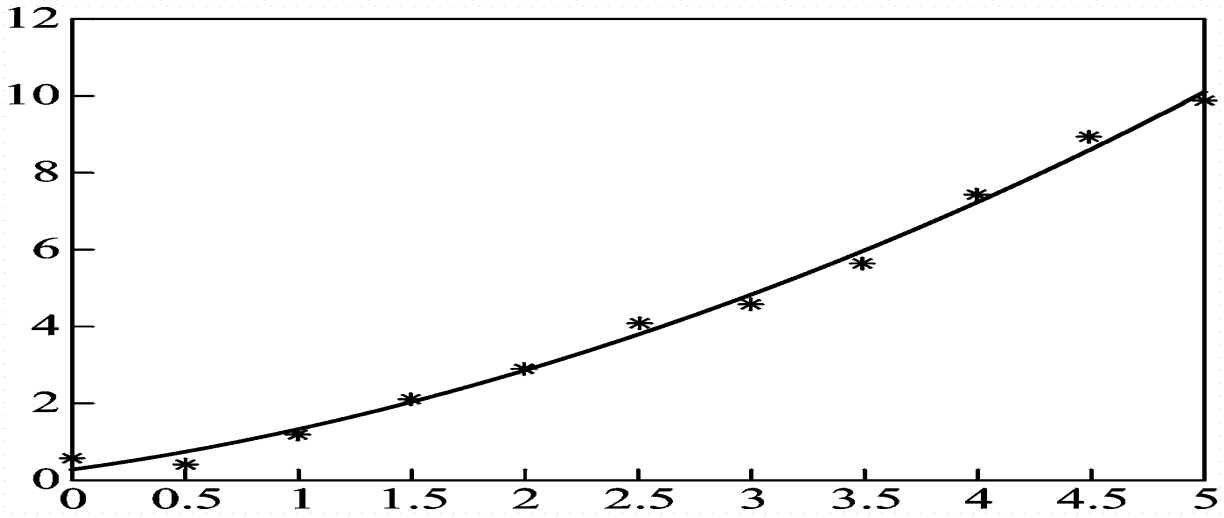
- polyfit (X,Y,N)： 最小二乘法多项式拟合
- polyval(P,X)： 计算多项式的值

X，Y： 数据点及其函数值

N: 拟合多项式的最高幂次

P： 被求值的多项式系数，按降幂排列

```
t=0:0.5:5;
s=1*t+0.2.*t.*t;                          %给定二次曲线
s1=[0.5  -0.18  -0.01  0.13  0.1 ...
 0.31  -0.22  -0.31  0.2  0.4  -0.14];
ss=s+s1;                                  %设定误差值
P=polyfit(t,ss,2);                        %二次曲线拟合
y=polyval(P,t);                           %计算拟合函数的值
plot(t,y,t,ss,'r*')
```



例 求下面数据表的最小二乘二次拟合多项式

i	1	2	3	4	5	6	7	8	9
x_i	−1	−0.75	−0.5	−0.25	0	0.25	0.5	0.75	1
y_i	0.2209	0.3295	0.8826	1.4392	2.0003	2.5645	3.1334	3.7601	4.2836

设拟合多项式：

$$P_2(x) = a_0 + a_1 x + a_2 x^2$$

正则方程组：

$$\begin{cases} 9a_0 + 0 + 3.75a_2 = 18.1732 \\ 0 + 3.75a_1 + 0 = 8.4842 \\ 3.75a_0 + 0 + 2.7656a_2 = 7.6173 \end{cases}$$

北京师范大学

方程组解：$a_0 = 2.0034, a_1 = 2.2625,\ a_3 = 0.0378$

所求多项式：$P_2(x) = 0.0378x^2 + 2.2625x + 2.0034$

用MATLAB验证结果

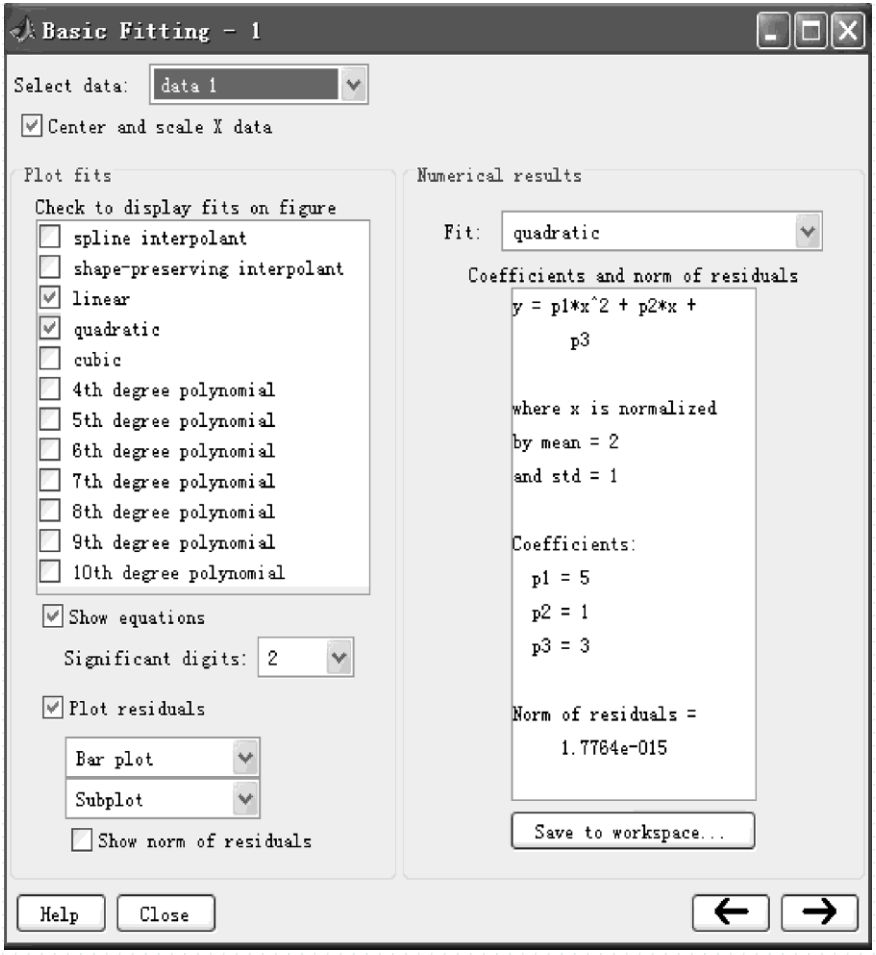
```
x=[-1 -0.75 -0.5 -0.25 0 0.25 0.5  0.75  1] ;
y=[-0.2209 0.3295 0.8826 1.4392 2.0003   2.5645
   3.1334 3.7601 4.2836];
yi=polyfit(x,y,2)
yi =  0.0379    2.2624    2.0034
```

§4.2.4 图形窗口的曲线拟合功能

优势：拟合快速、操作简便

功能：

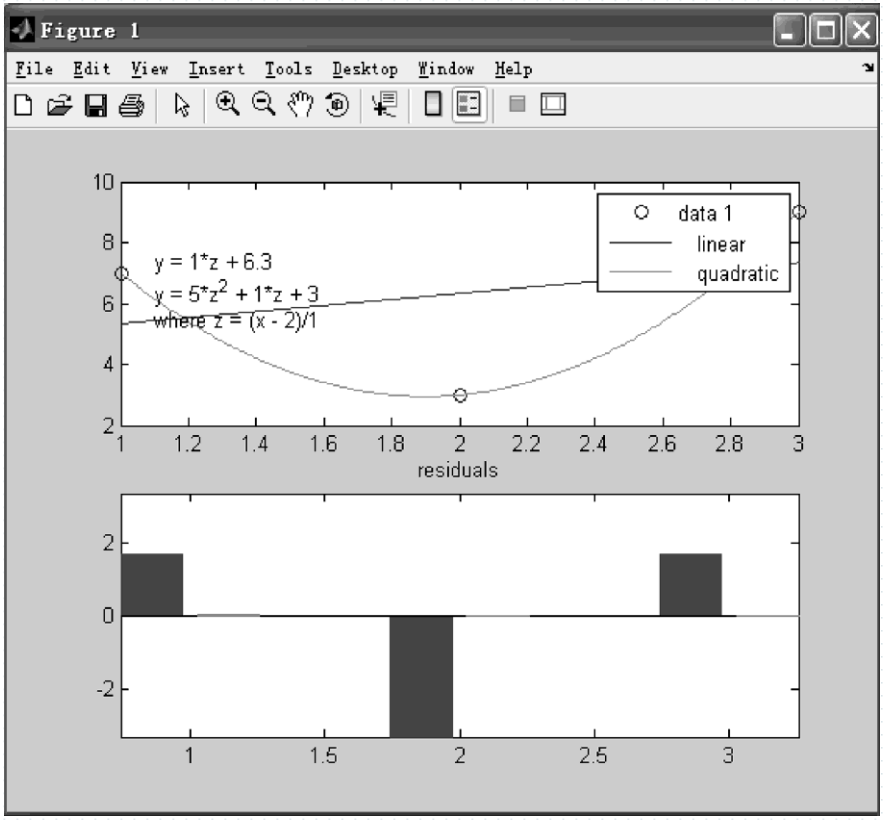
- 一组数据做多种拟合曲线，如样条拟合、保形拟合、1到10次的多项式拟合
- 绘制残差图
- 显示拟合曲线的表达式
- 用拟合方程和标准差注释图形
- 保存拟合曲线和计算结果



例

- x=[1,2,3];y=[7,3,9];plot(x,y,‘o’)

在指令窗输入数据画图，拟合数字结果与图形：



原始数据：圆圈

表达式：

$$y = z + 6.3$$

$$y = 5z^2 + z + 3$$

$z = (x-2)/1$是经过中心化和规范化的变量

§4.2.5 指数拟合

当数据点的分布近似于指数曲线

应考虑用指数函数$y = be^{ax}$拟合

先对上式取对数

$\ln y = ax + \ln b$

同时对数据y_i，也取对数得$\ln y_i$，利用数据组$(x_i, \ln y_i)$求出最小二乘拟合直线$\bar{y} = a_0 + a_1 x$

再取指数即得到原数据组的最小拟合指数

$y = e^{a_0 + a_1 x}$

例 一发射源的发射强度具有指数形式：$I = I_0 e^{-\alpha t}$

观测数据：

t_i	0.2	0.3	0.4	0.5	0.6	0.7	0.8
I_i	3.16	2.38	1.75	1.34	1.00	0.74	0.56

试用最小二乘法确定I_0与α

```
>> x=[0.2, 0.3,0.4,0.5,0.6,0.7,0.8];
>> I=[3.16,2.38 ,1.75,1.34,1.00,0.74,0.56];
>> y=log(I);
>> polyfit(x,y,1)
ans =
         -2.8883       1.7283
>> I0=exp(1.7283)
    I0 = 5.6311
>> alpha=-(-2.8883)
    alpha = 2.8883
```

强度公式为：
$I = 5.6311e^{-2.8883t}$

§4.2.6 线性最小二乘法的一般形式

- 已知n个数据点(x_i, y_i)，取$[a,b]$上线性无关的函数组$\varphi_0(x), \varphi_1(x), \cdots \varphi_m(x)$构成拟合函数

$$\varphi(x) = a_0\varphi_0(x) + a_1\varphi_1(x) + \cdots + a_m\varphi_m(x)$$

记偏差：

$$\delta_i = y_i - \varphi(x_i)$$

要求通过系数a_i的选择，使得加权以后的偏差的平方和$\sum_{i=1}^{n} \omega_i \delta_i^2$最小。

与多项式拟合的讨论相类似，正则方程组：

$$[\sum_{i=1}^{n} \omega_i [y_i - \varphi(x_i)]\varphi_j(x_i) = 0 (j = 0,1,2\cdots\cdots m)$$

即
$$\sum_{k=0}^{m} a_k \left[\sum_{i=1}^{n} \omega_i \varphi_k(x_i) \varphi_j(x_i) \right] = \sum_{i=1}^{n} \omega_i y_i \varphi_j(x_i)$$

如果记
$$(\varphi_k, \varphi_j) = \sum_{i=1}^{n} \omega_i \varphi_k(x_i) \varphi_j(x_i)$$

$$(y, \varphi_j) = \sum_{i=1}^{n} \omega_i y_i \varphi_j(x_i)$$

则方程组可以写成：

$$\begin{bmatrix} (\varphi_0,\varphi_0) & (\varphi_0,\varphi_1) & \text{L} & (\varphi_0,\varphi_m) \\ (\varphi_1,\varphi_0) & (\varphi_1,\varphi_1) & \text{L} & (\varphi_1,\varphi_m) \\ \text{M} & \text{M} & \text{M} & \text{M} \\ (\varphi_m,\varphi_0) & (\varphi_m,\varphi_1) & \text{L} & (\varphi_m,\varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \text{M} \\ a_m \end{bmatrix} = \begin{bmatrix} (y,\varphi_0) \\ (y,\varphi_1) \\ \text{M} \\ (y,\varphi_m) \end{bmatrix}$$

$\varphi_j(x)$的线性无关性 → 系数矩阵非奇异 → 方程组有唯一的解存在

通常选取$\varphi_j(x)$满足

$$(\varphi_j,\varphi_j)=\sum_{i=1}^{n}\omega_i\varphi_j^2(x_i)>0$$

$$(\varphi_k,\varphi_j)=0(k\neq j)$$

选取了正交函数族 ⇒ 系数矩阵成对角型

方程组解为 $a_j=\dfrac{(y,\varphi_j)}{(\varphi_j,\varphi_j)}=\dfrac{\sum_{i=1}^{n}\omega_i y(x_i)\varphi_j(x_i)}{\sum_{i=1}^{n}\omega_i\varphi_j^2(x_i)}$

最小二乘函数：$\varphi(x)=\sum_{j=0}^{m}a_j\varphi_j(x)=\sum_{j=0}^{m}\dfrac{(y,\varphi_j)}{(\varphi_j,\varphi_j)}\varphi_j(x_i)$

§4.3 曲线拟合工具箱(阅读)

步骤

1. 在指令窗口中输入数据

2. 用cftool启动曲线拟合工具箱，在对话窗口选择数据集，选择拟合公式自动拟合。

3. 必要时可对数据作预处理，以提高拟合的可靠性；拟合结束后可对结果作各种统计分析。

§4.4 求方程的零点

§4.4.1 求单调函数的实数根

1. 对分法

函数$f(x)$在区间$[a,b]$单调连续

$f(a)<0, f(b)>0$ ⟹

a,b间必有一实根

讨论 中点 $x_0=(a+b)/2$

- 若$f(x_0)=0$，则x_0为所求实根
- 若$f(x_0)<0$，则令$a_1=x_0, b_1=b$
- 若$f(x_0)>0$，则令$b_1=x_0, a_1=a$

结论：这时根必在$[a_1,b_1]$中，它的长度为原区间的一半。

- 令$x_1=(a_1+b_1)/2$再重复做，新区间$[a_2,b_2]$缩小一半。由此可得一区间系列

$$[a,b]\quad[a_1,b_1]\quad[a_2,b_2]\cdots[a_n,b_n]$$

- 当$n\to\infty$, $[a_n,b_n]$的长度$b_n-a_n=(b-a)/2^n$趋于零，且

$$x^*=\lim_{n\to\infty}x_n=\lim_{n\to\infty}\frac{a_n+b_n}{2}$$

是方程根的近似值。

特点

- 根据需要的精度来决定计算次数
- 优点：可靠
- 缺点：收敛速度慢

2. 切线法(比对分法速度快)

Taylor展开的一级近似

$$f(x_n)+f'(x_n)(x-x_n)=0$$

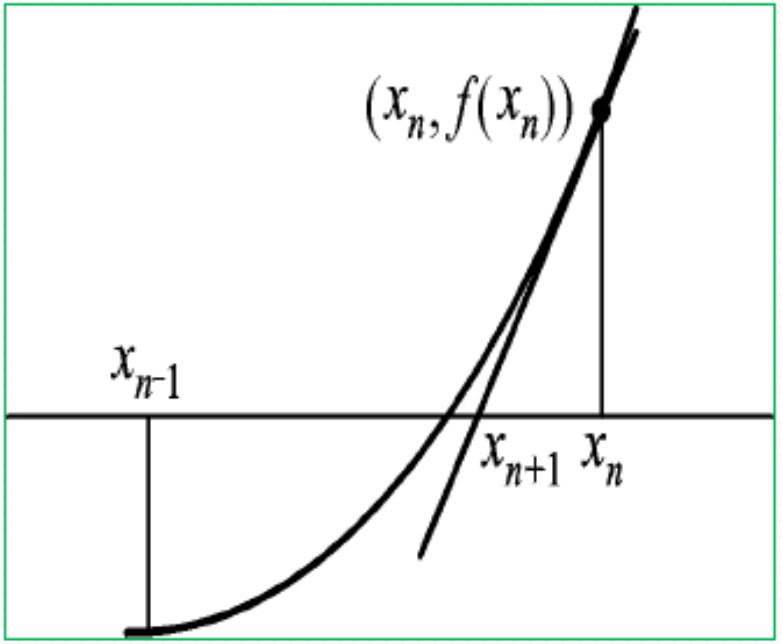
设$f'(x_n)\neq 0$,
以x_{n+1}代替x得到牛顿迭代序列为

$$x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}$$

x_{n+1}是过点 $(x_n,f(x_n))$的切线

$$y-f(x_n)=f'(x_n)(x-x_n)$$

与x轴的交点，所以叫切线法。



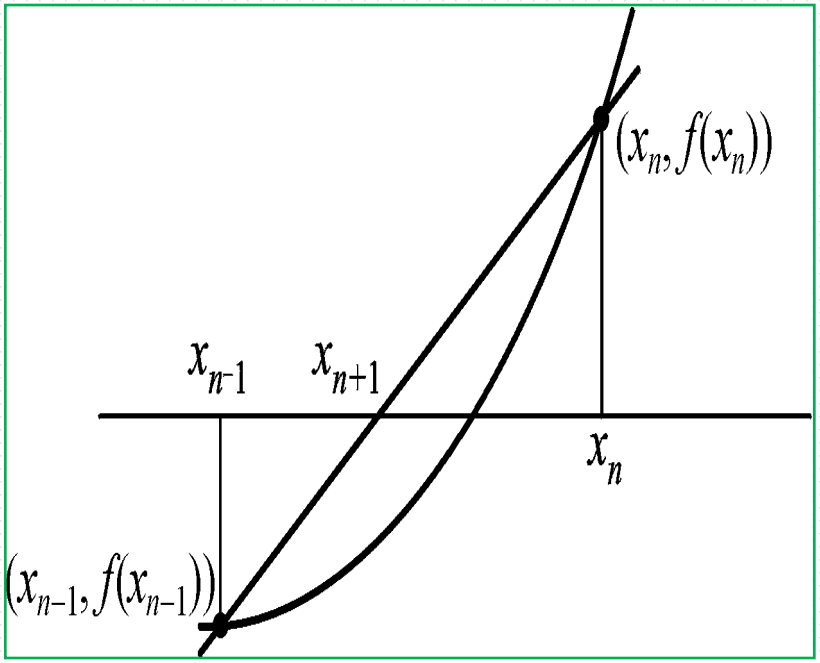
3. 弦割法

用差分公式代替上式中的导数

$$f'(x_n)=\frac{f(x_n)-f(x_{n-1})}{x_n-x_{n-1}}$$

得递推公式

$$x_{n+1}=x_n-f(x_n)\frac{(x_n-x_{n-1})}{f(x_n)-f(x_{n-1})}$$



说明：

（1）用过点$(x_{n-1}, f(x_{n-1}))$，$(x_n, f(x_n))$的直线与x轴的交点来求根。

（2）优点与缺点和牛顿法一样，但不需要直接计算 $f'(x)$。

4. 求函数值对应的自变量值

问题：

指定函数值η，求对应的自变量的值。

解决方法：

建立如下函数

$$F(x)=f(x)-\eta$$

然后再求$F(x)$的零点。

北京师范大学

§4.4.2 MATLAB解方程的指令

1.fzero：求单变量函数的零点

```
x=fzero(fun, x0)
x=fzero(fun,x,options,p1,p2,…)
[x,fval]=fzero(……)
```

x, fval	零点及对应的函数值，没有零点则返回NaN
fun	求零点的单变量函数，函数值须是实数
x0	猜测的初始值或搜寻零点的区间[x0(1),x0(2)]
options	控制算法的优化选项，[]是默认值

- 使用fzero要输入的两个基本参数:
  1）要求解的函数； 2）初始的猜测解或求解的区间。
- 函数的函数（function functions）：将函数当作参数用

一个函数作为参数传递给另一个函数，有三种方法可用：

（1）函数句柄:@+函数名

```
x = fzero(@sin,3)
x = 3.1416
```

```
function ff = myfun(x,a)
ff= cos(a*x);
a=2;                                  %先给出参数a的值再求根
x = fzero(@(x) myfun(x,a),0.1)        %调用时要注明自量
x =  0.7854
```

对于带参数的内部函数也可以用这种方法调用
如:besselj是求贝塞尔函数值的内部函数。

```
>> fzero(@(x)besselj(0,x),[0,pi])
ans = 2.4048
```

（2）匿名函数

函数表达式直接作为参数写入，而不定义函数名称。

```
>> fzero(@(x)5*x^5+3*x^4-4*x.^3+x.^2+7*x+9,1)
ans = -1.3127
```

（3）内嵌对象

MATLAB7版本之前采用的一种定义函数方法。

```
>> f=inline('5*x^5+3*x^4-4*x.^3+x.^2+7*x+9');
>> fzero(f,1)
ans =  -1.3127
```

推荐使用函数句柄

```
>> f=@(x)5*x^5+3*x^4-4*x.^3+x.^2+7*x+9
f =  @(x)5*x^5+3*x^4-4*x.^3+x.^2+7*x+9
>> X=fzero(f,1)
X = -1.3127
```

- 带参数的函数也能这样定义：

```
>> B=@(x,n,y)besselj(n,x)-y
    B =  @(x,n,y)besselj(n,x)-y
>> B(1,0,0.5)
    ans = 0.2652
```

用fzero调用：

```
>> fzero(@(x)B(x,0,0.5),[0,2])
    ans = 1.5211
```

2. 求多项式的零点roots(c)全部零点

例 求$5x^5+3x^4-4x^3+x^2+7x+9=0$的零点。

```
roots([5,3,-4,1,7,9])

ans = 0.95299 +    0.76221i
      0.95299 -    0.76221i
     -1.3127
     -0.59662 +    0.75154i
     -0.59662 -    0.75154i
```

系数按降幂排列

$c=[c_n,c_{n-1},\cdots,c_1,c_0]$

3. 解非线性方程组fsolve

用法	x = fsolve(fun, x0) [x,fval]=fsolve(fun,x0, ...)

x, fval	零点的位置与对应的函数值
fun	求解的非线性方程组
x0	猜测的初始解

例

$$\begin{cases} 2\cos(x)+\sqrt{y}-\ln(z)=7 \\ 2^x+2y-8z=-1 \\ x+y-\cosh(z)=0 \end{cases}$$

改写成

$$\begin{cases} Y(1)=2\cos(x)+\sqrt{y}-\ln(z)-7 \\ Y(2)=2^x+2y-8z+1 \\ Y(3)=x+y-\cosh(z) \end{cases}$$

➢ 编写函数文件qg.m

```
function Y = qg(X)
Y = [2*cos(X(1)) + sqrt(X(2)) - log(X(3)) - 7;
     2^X(1) + 2*X(2) - 8*X(3) + 1;
     X(1) + X(2) - cosh(X(3))]
```

➢ 猜测初始解x_0　　$x_0 = [0.5 + 0.5i, 0.5 + 0.5i, 0.5 + 0.5i]$

➢ 求解

```
>>[X,Y]=fsolve('qg',[0.5+0.5i, 0.5+0.5i, 0.5+0.5i])
X = -0.014456  +    1.601i
      1.0567   -    1.7481i
      0.44424  -    0.3262i
Y = -1.2777e-011   +1.2092e-011i
      1.5534e-012  +5.4401e-013i
      1.199e-014   +1.3936e-013i
```


北京师范大学

4. 解符号变量方程solve

（注意与数值解的区别）

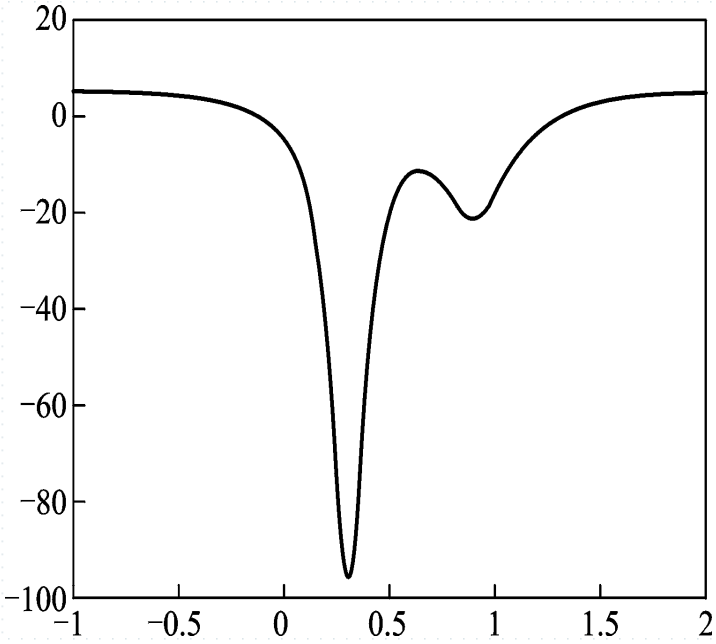
- syms x e
- v2=e-4*(x.^(-12)-x.^(-6))
- solve(v2)
- %运算结果得到12个根的表达式
-

北京师范大学

§4.4.3 搜寻实函数的极小值fminbnd

用画图判断函数有无极小值存在

```
>> f =@(x)-1 ./ ((x-.3).^2 + .01) - 1 ./ ((x-.9).^2 + .04) + 6 ;
>> x=-1:0.01:2; y=f(x);
>> plot(x,y)
```



极小值在区间[0,0.4]中

```
>> fminbnd(f,0,0.4)
ans =  0.30038
```

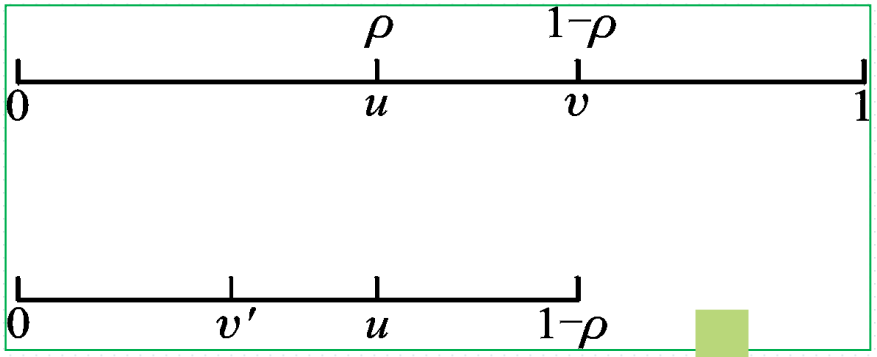
说明：

- fminbnd只能找一个极小值。
- 对于多变量的单值函数，用指令fminsearch搜寻它的局部极小值。

fminbnd的算法：黄金分割法与抛物线插值法

（1）黄金分割法

取$a=0, b=1$，
$h=\rho(b-a)$，ρ比1/3略大。点$u=a+h$和$v=b-h$将区间分成三个



如果有$f(u)<f(v)$，就可以用v代替b，为了$f(u)$能继续使用，ρ需满足：

$$\frac{\rho}{1-\rho}=\frac{1-\rho}{\rho}$$

解得$\rho=(3-\sqrt{5})/2=2-\phi=0.382$

其中$\phi=(1+\sqrt{5})/2$，另一新点u'：$\rho(1-\rho)$

区间长度每步缩小比例：

$$\phi - 1 \approx 0.618$$

若干步后达到eps的量级即2^{-52}，就有

$$\frac{-52}{\log(\phi - 1)} \approx 75$$

（2）抛物线插值法

几次计算后得到区间内三个不同的点及其函数值。

如果由这三个点插值产生的抛物线的最小值在这个区间内。



那么下一个点通常是选这个极小值而不是选黄金分割点。

黄金分割与抛物线插值相结合，提供了一个一维化问题的可靠而有效的求解方法。

§4.5 快速傅里叶变换

§4.5.1 离散傅里叶变换

令$\omega_n = e^{-i2\pi/n}$（1的n次根），则$\omega_n^n = 1$

N个数据点$x_l(l = 0,1,\ldots N-1)$的离散傅里叶变换

$$y_m = \sum_{k=0}^{n-1} x_k \omega_n^{mk}，（m = 0，1，2\ldots n-1）$$

用矩阵表示

$$y = F_n x$$

$n \times n$的矩阵F的矩阵元F_{mk}记为

$$\{F_n\}_{mk} = \omega_n^{mk}$$

当$n=4$时，$\boldsymbol{F}_4=\begin{bmatrix}1&1&1&1\\1&\omega^1&\omega^2&\omega^3\\1&\omega^2&\omega^4&\omega^6\\1&\omega^3&\omega^6&\omega^9\end{bmatrix}=\begin{bmatrix}1&1&1&1\\1&-i&-1&i\\1&-1&1&-1\\1&i&-1&-i\end{bmatrix}$

利用 $\omega_n^1=-i,\omega_n^2=-1,\omega_n^3=i,\omega_n^4=1$

得 $\frac{1}{n}\begin{bmatrix}1&1&1&1\\1&\omega^{-1}&\omega^{-2}&\omega^{-3}\\1&\omega^{-2}&\omega^{-4}&\omega^{-6}\\1&\omega^{-3}&\omega^{-6}&\omega^{-9}\end{bmatrix}\begin{bmatrix}1&1&1&1\\1&\omega^1&\omega^2&\omega^3\\1&\omega^2&\omega^4&\omega^6\\1&\omega^3&\omega^6&\omega^9\end{bmatrix}$

$=\begin{bmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\end{bmatrix}$

北京师范大学

F_4的逆矩阵为 $$F_4^{-1}=\frac{1}{n}\begin{bmatrix}1 & 1 & 1 & 1\\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3}\\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6}\\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9}\end{bmatrix}$$

以此类推，傅里叶变换的逆变换为

$$x=F_n^{-1}F_nx=F_n^{-1}y$$

即 $$x_k=\frac{1}{n}\sum_{m=0}^{n-1}y_m\,\omega_n^{-mk},\quad k=0,1,\ldots,n-1$$

从表达式$\omega_n^{-mk}=\left(e^{-i\frac{2\pi}{n}}\right)^{-mk}=e^{ik\frac{2\pi}{n}m}$，可知

- k/n相当于频率。
- $2\pi k/n$相当于圆频率。
- y_m/n相当于各频率的振幅，振幅的平方称为功率谱。

说明

- y_0：零频分量是X的各个分量之和，是常数。
- $y_{n/2}$：尼奎斯特频率，是采样频率的一半，是由取样频率所表达的最高频率。
- 由于离散傅里叶变换在频率上有周期性，所以除零频及尼奎斯特频率以外的频率两两共轭的。

北京师范大学

§4.5.2 傅里叶级数与傅里叶积分（阅读）

离散傅里叶变换：将N个的离散的数据变换成相同长度的离散的频谱系列。

傅里叶级数：将变量范围为无限的连续的周期函数变换成无限的离散的傅里叶频谱序列。

傅里叶积分：对变量范围为无限的连续的非周期函数变换成无限的连续的傅里叶频谱序列。

离散时间傅里叶变换：将无限长数据序列变换为周期函数。

书上这些内容仅供有兴趣的读者参考。

1. 傅里叶级数

定义在 $[-l,l]$ 上的任意平方可积函数 $f(x)$ 可展成傅里叶级数

$$f(x)=C_0+\sum_{n=1}^{\infty}\left(C_n\cos\frac{n\pi x}{l}+D_n\sin\frac{n\pi x}{l}\right)$$

$f(x)$ 也可以展成如下复数形式的傅里叶级数

$$f(x)=\sum_{n=-\infty}^{\infty}C_n e^{i\frac{n\pi x}{l}}$$

$$C_n=\frac{1}{2l}\int_{-l}^{l}f(x)e^{-i\frac{n\pi x}{l}}\mathrm{d}x$$

2. 傅里叶积分

$$G(\omega)=\frac{1}{2\pi}\int_{-\infty}^{\infty}g(x)e^{-i\omega x}\cdot\mathrm{d}x$$

$$g(x)=\int_{-\infty}^{\infty}G(\omega)e^{i\omega x}\cdot\mathrm{d}\omega$$

北京师范大学

§4.5.3 快速傅里叶变换的指令

- 快速傅里叶变换(FFT)是离散傅里叶变换(DFT)的快速算法。

MATLAB离散傅里叶变换使用的公式：

$$X(k)=\sum_{n=1}^{N}x(n)e^{j2\pi(k-1)\left(\frac{n-1}{N}\right)}$$

$$x(n)=\frac{1}{N}\sum_{k=1}^{N}X(k)e^{-j2\pi(k-1)\left(\frac{n-1}{N}\right)}$$

频率：$2\pi(k-1)/N$ ；各频率的振幅：$X(k)/N$

1.一维的FFT及其逆变换

指令及语句格式：fft(x)　　fft(x, N)

说明：

- x：待变换的数据
  - 输入数据的长度是2的整次幂，则FFT自动采用以2为基数的快速算法，否则就采用较慢的算法。
  - 若x是矩阵，FFT是对每一列进行。
- N：进行N个点的FFT
  - 如果N大于x的长度,则用零补充。
  - 如果x大于N的长度,则把多余的部分去掉。
- 逆变换ifft的格式与fft的格式相同

北京师范大学

2. 二维的FFT及其逆变换

- 指令及其语句格式：fft2(x)

说明：

- x：要变换的二维矩阵。如果它是一维的，则按一维进行变换。
- 逆变换ifft2的格式与fft2的格式相同。
- n维的FFT及其逆变换指令fftn。ifftn的格式分别与fft2,ifft2的格式相同。

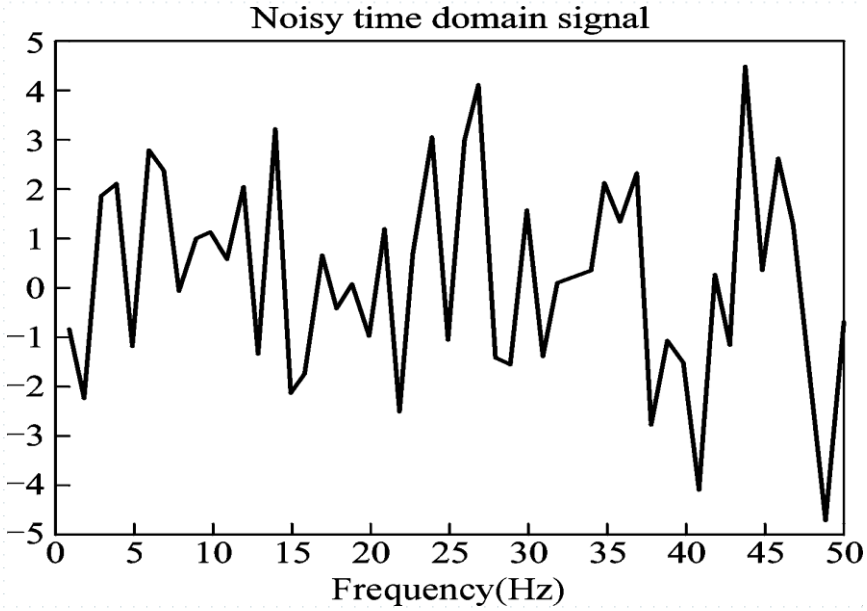
例　一维傅里叶变换

```
>>X = [ 4, 0, 3, 6, 2, 9, 6, 5] ;
>>Y=fft(X)
Y = 35                                          %零频分量
 -5.0711 +8.6569i
 -3 +       2i
 9.0711 +2.6569i
 -5                                             %尼奎斯特频率分量
 9.0711 - 2.6569i
 -3         - 2i
 -5.0711 -   8.6569i
```

共轭分量

例

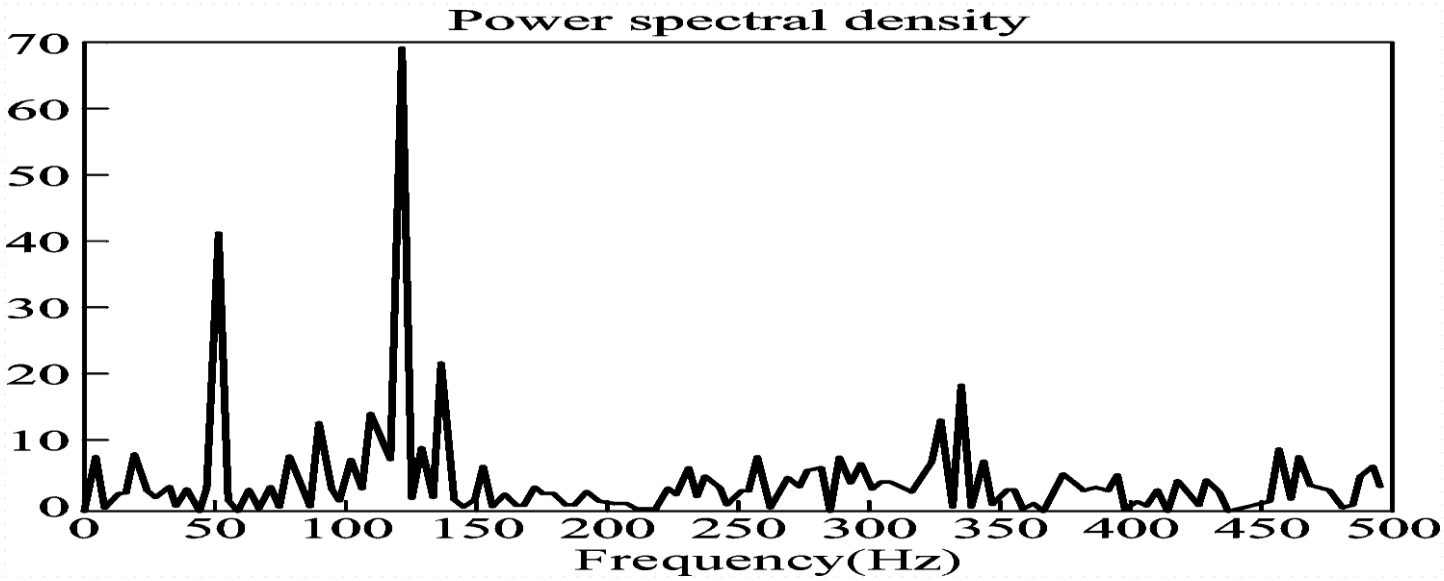
- 取样频率1000Hz,
- 取样间隔1/1000s=1ms
- 取样数250
- 总取样时间0.25S
- 建立时间轴：从0到0.25s
- 构造正弦波的信号
  包含频率50Hz与100Hz




```
t = 0:0.001:.25;
x = sin(2*pi*50*t) + sin(2*pi*120*t);
y = x + 2*randn(size(t));                    %噪声信号
plot(y(1:50))
title('Noisy time domain signal')
```

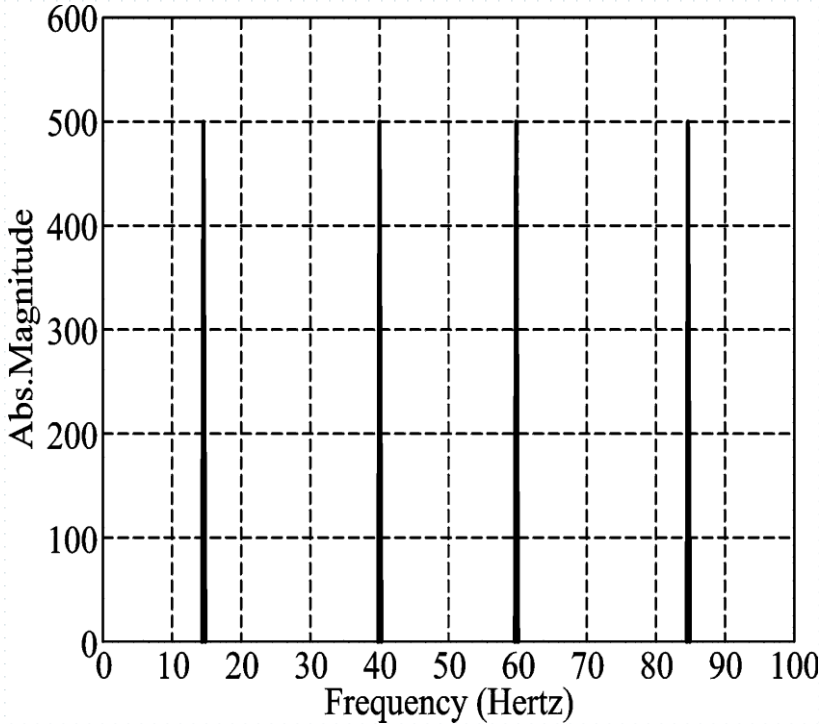
对噪声信号y作FFT

```
Y = fft(y,256);
Pyy = Y.*conj(Y)/256;
f = 1000/256*(0:127);
plot(f,Pyy(1:128))
title('Power spectral density')
xlabel('Frequency (Hz)')
```



例 对函数$x = \sin(30\pi t) + \sin(80\pi t)$作快速傅里叶变换。

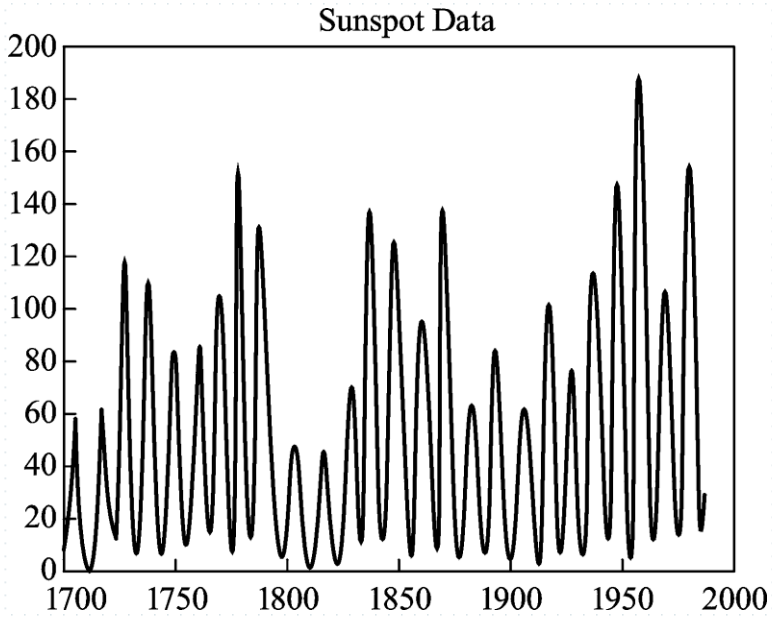
```
t = 0:1/100:10-1/100;
x = sin(2*pi*15*t) +sin(2*pi*40*t);
y = fft(x);
m = abs(y);
f =(0:numel(y)-1)'*100/numel(y);
plot(f,m)
grid on
ylabel('Abs. Magnitude')
xlabel('Frequency [Hertz]')
```



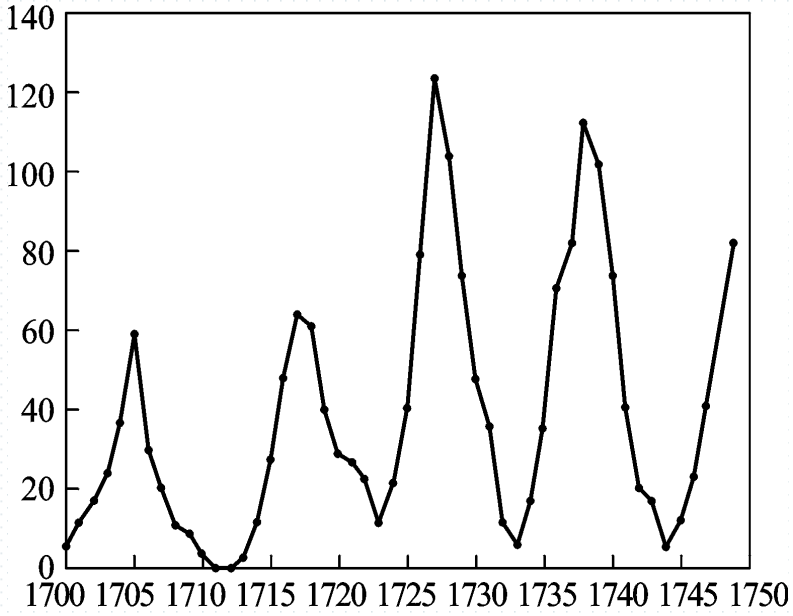
指令numel：计算列阵中的元素总数。

例　分析太阳黑子活动的周期性

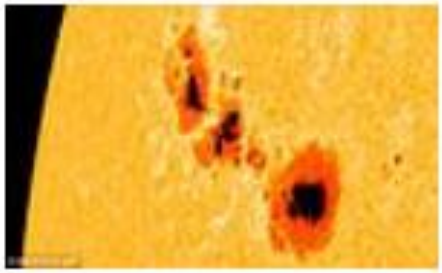
通过分析300年左右的太阳黑子活动的记录资料可以发现每过11年就会达到一个极大值。



Wolfer数图：
太阳黑子的数目与面积



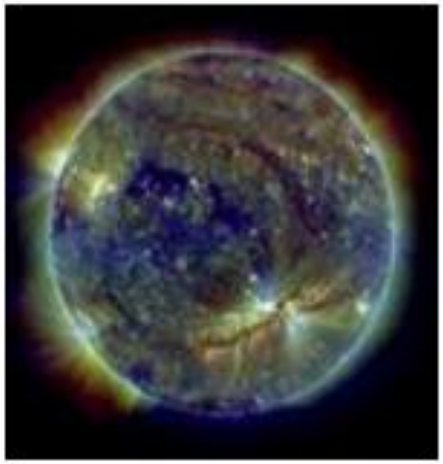
放大的前50年的数据



2011年9月的太阳黑子。



2004年6月22日的太阳黑子影像。



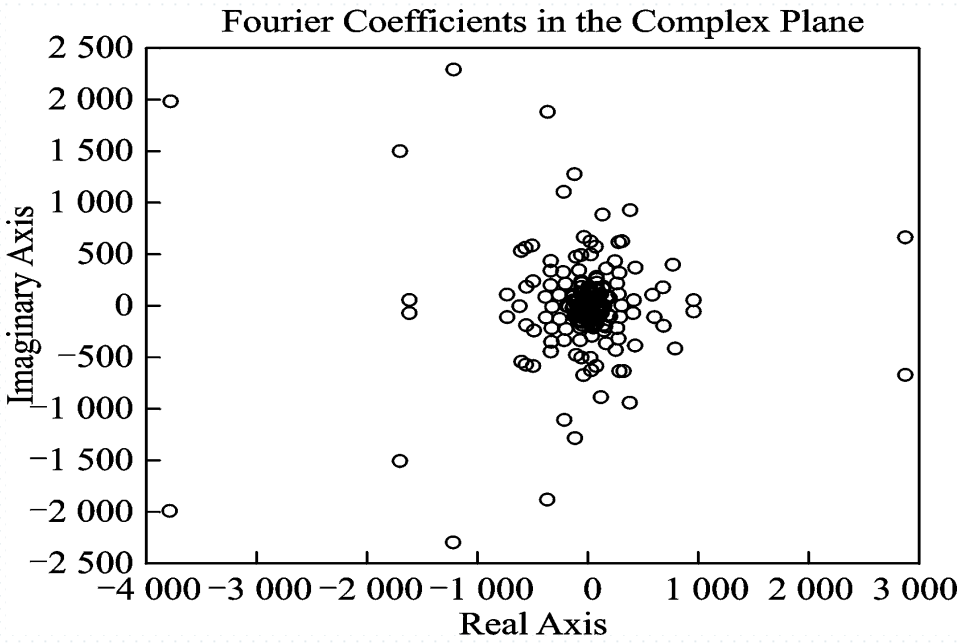
2010年10月在不同黑子上方的看见的日冕构造。

图片取自维基百科

用FFT对太阳黑子数据处理:

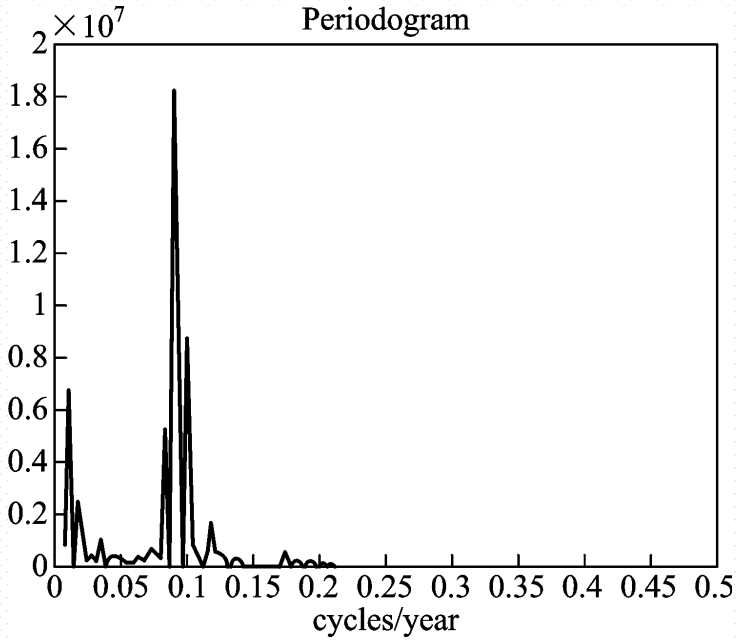
- Y = fft(wolfer); Y(1)=[];
- plot(Y,'ro')
- title('Fourier Coefficients in the Complex Plane');
- xlabel('Real Axis');
- ylabel('Imaginary Axis');

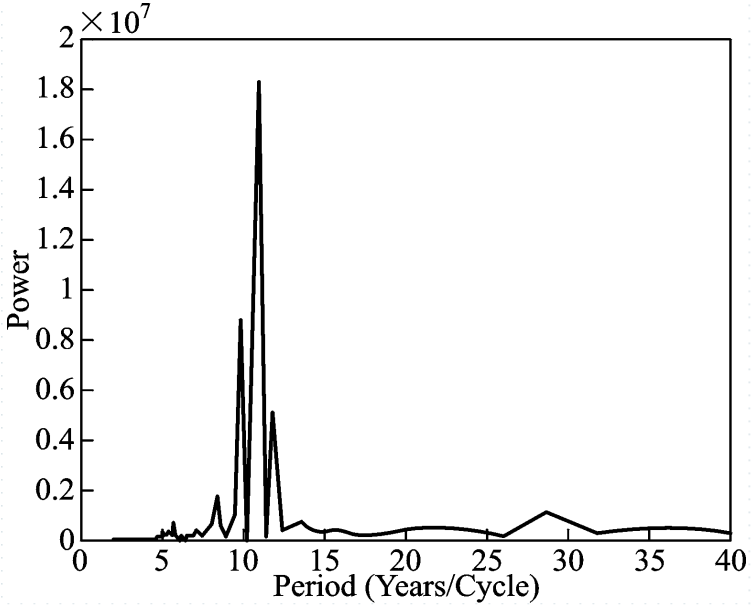
傅里叶系数Y在
复平面上的分布



- 画振幅相对于频率的周期图

```
n=length(Y);
power = abs(Y(1:floor(n/2))).^2;
freq = (1:n/2)/n;
plot(freq,power)
xlabel('cycles/year')
title('Periodogram')
```





将周期图的横坐标改变为倒数，即将(周期/年)改为（年/周期）可以清楚看出11年的周期。

§4.5.4 快速傅里叶变换的算法（阅读）

- FFT：利用离散傅里叶变换的对称性和允余性的快速算法。

以$n=4$为例说明：

$$y_m=\sum_{k=0}^{3}x_k\omega_n^{mk},(m=0,1,2,3)$$

$$y_0=x_0\omega_n^0+x_1\omega_n^0+x_2\omega_n^0+x_3\omega_n^0$$

$$y_1=x_0\omega_n^0+x_1\omega_n^1+x_2\omega_n^2+x_3\omega_n^3$$

$$y_2=x_0\omega_n^0+x_1\omega_n^2+x_2\omega_n^4+x_3\omega_n^6$$

$$y_3=x_0\omega_n^0+x_1\omega_n^3+x_2\omega_n^6+x_3\omega_n^9$$

- 注意到：

$$\omega_n^0=\omega_n^4=1,\omega_n^2=\omega_n^6=-1,\omega_n^9=\omega_n^1$$

重新组合

$$y_0=(x_0+\omega_n^0x_2)+\omega_n^0(x_1+\omega_n^0x_3)$$
$$y_1=(x_0-\omega_n^0x_2)+\omega_n^1(x_1-\omega_n^0x_3)$$
$$y_2=(x_0+\omega_n^0x_2)+\omega_n^2(x_1+\omega_n^0x_3)$$
$$y_3=(x_0-\omega_n^0x_2)+\omega_n^3(x_1-\omega_n^0x_3)$$

- 现在要8次加法和6次乘法，
  原来要12次加法和16次乘法。
- 4点的DFT变成了2点的对奇数数列和偶数数列的变换。

若n是偶数,则n点的DFT可能分解成两个长度为一半的DFT。

回顾：几个傅里叶矩阵

$$F_1 = 1, F_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}$$

引入置换矩阵$\boldsymbol{P}_4$和对角矩阵$\boldsymbol{D}_2$

$$\boldsymbol{P}_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{D}_2 = \begin{bmatrix} 1 & 0 \\ 0 & -i \end{bmatrix}$$

可得

$$F_2 P_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & -i & i \\ 1 & 1 & -1 & -1 \\ 1 & -1 & i & -i \end{bmatrix} = \begin{bmatrix} F_2 & D_2 F_2 \\ F_2 & -D_2 F_2 \end{bmatrix}$$

前面交换以后的结果是$\boldsymbol{F_4 P_4 x}$。

北京师范大学

总结：

- F_4可以排列成用F_2组成的方块。
- 点数是偶数，分层分解的步骤就可以逐层进行下去。

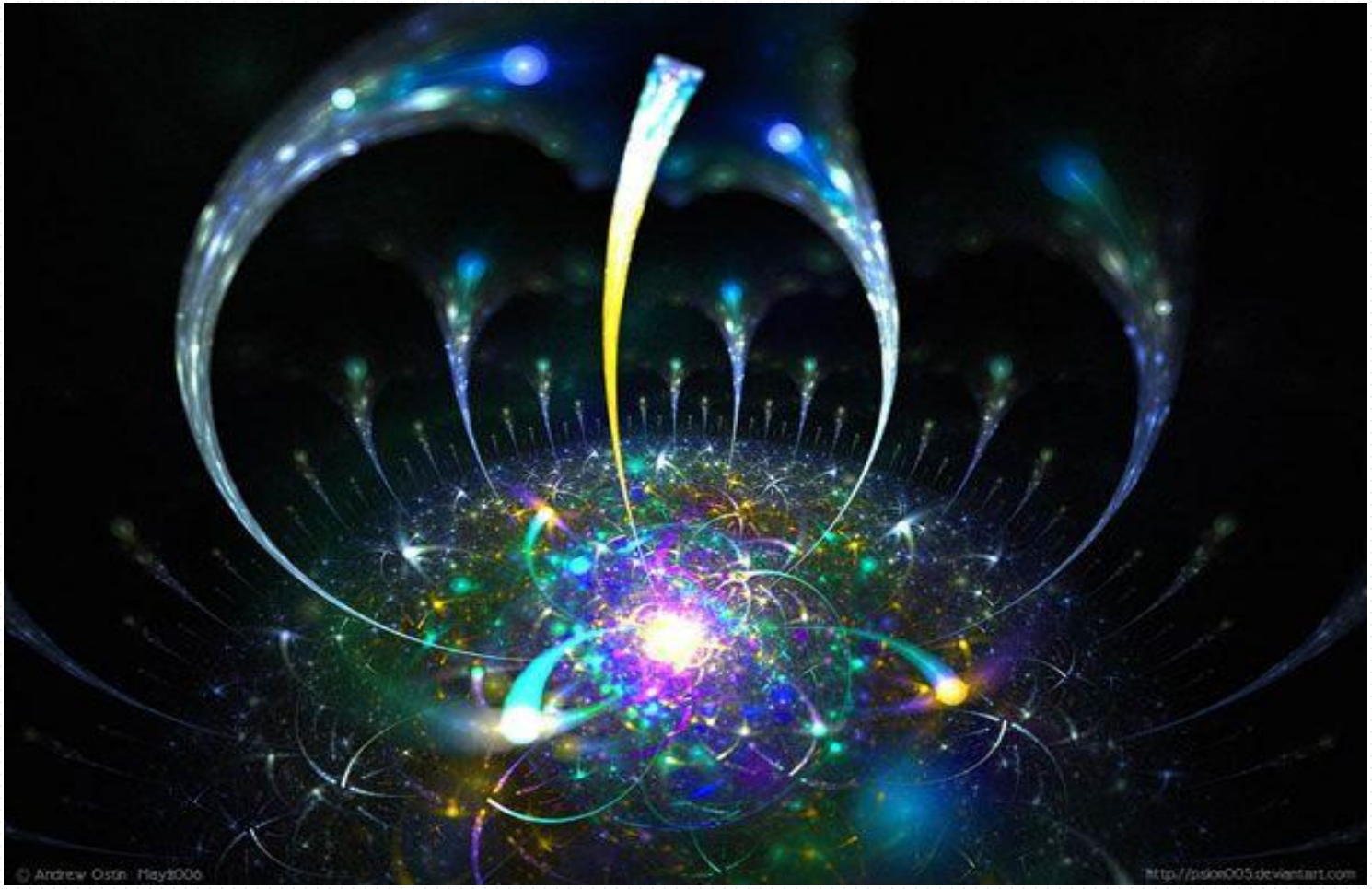
F_4：将F_n的偶数列放到奇数列之前。

$$D_{n/2} = \mathrm{diag}\left(1, \omega_n, \ldots, \omega_n^{(n/2)-1}\right)$$

F_n应用于长度为n的数列：

1. 将$F_{n/2}$分别作用于它的偶数列和奇数列。
2. 用$D_{n/2}$标度一下。

用这种递归分解与解决的方法来计算DFT就叫FFT。



第四章完

北京师范大学